MixPI: User Guide

Britta A. Johnson[1], Siyu Bu[2], Christopher J. Mundy[1], and Nandini Ananth[2]

[1]Physical Science Division, Pacific Northwest National Laboratory, Richland, Washington 99352, USA
[2] Department of Chemistry and Chemical Biology, Cornell University, Ithaca, New York 14850, USA

April 20, 2026

Contents

1 Downloading and Compiling CP2K and MixPI **2**
1.1 Download . . . 2
1.1.1 MixPI . . . 2
1.1.2 CP2K . . . 2
1.2 Compile . . . 3
1.2.1 CP2K . . . 3
1.2.2 MixPI . . . 3
1.2.3 Troubleshooting . . . 4
1.2.3.1 Cannot Locate Specific External Libraries . . . 4
1.2.3.1.1 ELPA . . . 4
1.2.3.1.2 Other External Libraries . . . 4

2 Running MixPI **4**
2.1 Input . . . 4
2.1.1 MixPI Input Files for PI Specific Information: ***rp_molecule_info.inp*** . . . 5
2.1.2 Input Generation Script: ***make_pimd_input.F*** . . . 5
2.1.3 CP2K Input Files . . . 6
2.1.3.1 Force Information in FORCE_EVAL: ***cp2k.inp*** . . . 7
2.1.3.1.1 Lennard-Jones Interactions . . . 7
2.1.3.1.2 Bonded Interactions . . . 8
2.1.3.1.3 Electrostatic Interactions . . . 9
2.1.3.2 Coordinate Information: ***structure.xyz*** . . . 10
2.1.3.3 Topology Information: ***structure.psf; rp_molset.dat; and classical_molset.dat*** . . . 10
2.1.3.3.1 PSF File . . . 11
2.1.3.3.2 MOL_SET Information . . . 13
2.1.4 MixPi Input Files for Molecular Dynamics Inputs . . . 13
2.1.4.1 ***md_run.inp*** . . . 13
2.1.4.2 ***file_info.inp*** . . . 15
2.1.4.3 System-Specific Input Files: ***xx_rcut.inp***, ***md_constraint.inp*** . . . 16
2.1.4.3.1 Systems with PI Electrons: Changes to ***cp2k.inp*** and ***xx_rcut.inp*** 16
2.1.4.3.2 Systems with Constraints: ***md_constraint.inp*** . . . 16
2.2 Output Files . . . 17
2.2.1 Energy Files . . . 17
2.2.2 Trajectory Files . . . 18
2.2.3 Final Configuration Files and RESTARTs . . . 19
2.2.4 Other Files . . . 19

3 Analysis Scripts **19**
3.1 RDF calculation 19

4 Theory **20**
4.1 All Replica Ring Polymer Hamiltonian 21
4.2 MixTS Treatment of 1/N 22
4.3 MTS-Expansion in Three-Dimensions 23

5 Funding Acknowledgment **25**

1 Downloading and Compiling CP2K and MixPI

1.1 Download

1.1.1 MixPI

The most recent version of MixPI is v2.1 and is available on the GitHub page:
https://github.com/AnanthGroup/Path-Integral-Atomistic-Driver [3]. Subsequent versions will be developed and updated to this page. The following subdirectories are located within the MixPI download:

- **src**: Contains the source codes for MixPI.
- **examples**: Example input files for a mixTS simulation of a metal (M^{3+}) cation + an explicit, quantized path integral (PI) e^- solvated in bulk water.
- **cp2k-updated-files**: The CP2K source files that were updated to interface with MixPI

All files in the **src** folder should be downloaded. The folder **updated-cp2k-files** contains the CP2K source files that were modified to work with MixPi and must be downloaded and placed in the **cp2k/src** folder as described below.

1.1.2 CP2K

MixPI requires the installation of the molecular dynamics package CP2K v2023.2 [5] which can be found on the CP2K website. CP2K must be installed as both an executable and library following the instructions on the CP2K website.

The following CP2K files, located in **cp2k/src** unless otherwise indicated must be replaced with modified versions to run MixPI simulations:

- ***constraint.F***
- ***constraint_vsite.F***
- **start/*libcp2k.F***
- **subsys/*molecule_kind_types.F***
- ***topology_connectivity_util.F***
- ***topology_coordinate_util.F***
- ***topology_generate_util.F***
- ***topology_psf.F***
- ***topology_types.F***
- ***topology_util.F***

The modified files are located in **cp2k-updated-files**. These changes do not interfere with running standard, classical MD simulations in CP2K.

1.2 Compile

1.2.1 CP2K

After replacing the updated files from **cp2k-updated-files** into the appropriate directory in the **src** folder in the CP2K build, the authors suggest using the toolchain build to generate arch files and compile CP2K. Instructions for the toolchain build are found in the CP2K documentation. To build the CP2K version used by the authors on a Linux OS, the following steps were used.

- From the **cp2k** directory, access the toolchain.
  cd /tools/toolchain

- Make sure you have updated paths to the gcc compiler and OpenMPI (we used gcc 10.3.0; OpenMPI 4.2.1). If not, it will download and install it's own versions.

- Run install command.
  ./install_cp2k_toolchain.sh
  ./install_cp2k_toolchain.sh –with-sirius=no
  For additional features and libraries to be linked to CP2K (like PLUMED), include those in the install line. (For example *./install_cp2k_toolchain.sh -with-plumed*)

- After the build, copy the arch files generated in **install/arch** to the CP2K **arch** directory
  cp install/arch/local ../../arch*

- Source the locations of the libraries that are linked to CP2K
  source install/setup

- Return to the cp2k directory and compile.
  cd ../..
  make -j 12 ARCH=local VERSION='ssmp sdbg psmp pdbg'

- After successful compilation, compile cp2k as library.
  make -j 12 ARCH=local VERSION='ssmp sdbg psmp pdbg' libcp2k

- The executables will now be located in **cp2k/exe/local**.

Directions for a toolchain build on an M1 Mac can be found on the CP2K website under "Compile CP2K on MacOS."

1.2.2 MixPI

After compiling CP2K as an executable and library, the paths in the makefile for MixPI must be updated to find the correct libraries. This is done with three separate updates:

- Copy all libraries flags from the CP2K arch file (The arch file is ***arch/local.psmp*** and the flags are located following "LIBS=".) onto the "LIB =" line in the makefile located in the **src** directory of MixPI.

- For MixPI to find these libraries, update the -I and -L paths within the makefile or, if compiled with the toolchain build, source the same setup file as used when building CP2K.
  *source /**cp2k-directory**/tools/toolchain/install/setup*

- Update the locations of the cp2k and dbcsr libraries and object files. They are located at:
  CP2K_LIB = **/**cp2k-directory**/lib/local/psmp**
  DBSCR_LIB = **/**cp2k-directory**/lib/local/psmp/exts/dbcsr**
  CP2K_INC = **/**cp2k-directory**/obj/local/psmp**
  DBSCR_INC = **/**cp2k-directory**/obj/local/psmp/exts/dbcsr**

This should generate an executable ***cp2k_pimd.out*** to run MixPI. An example makefile can be found in the **src** directory.

1.2.3 Troubleshooting

Some users have experienced a few specific issues with the instructions above. The most common issues (and their solutions) are outlined below.

1.2.3.1 Cannot Locate Specific External Libraries When using the toolchain build above, CP2K will automatically download, install, and link to various external libraries. Of these, the majority are optional and are only used for specific simulations in CP2K. A few of the prerequisites that are automatically generated in the toolchain have generated errors for some users; these include:

1.2.3.1.1 ELPA The ELPA libarary replaces the SCALAPACK matrix diagonalization in some CP2K subroutines. Some users report an error when compiling CP2K that it cannot locate the ELPA libaries. When this happens, the path that points towards the ELPA libararies is not generated correctly in the toolchain build. This can be fixed in two ways. The toolchain build can exclude the ELPA libarary by using the command
./install_cp2k_toolchain.sh –with-elpa=no
The user can also modify the install/setup file to point to the location of the include and library directories for ELPA. In most cases, this involves changing the paths in the setup file from:
cp2k/tools/toolchain/install/elpa-2022.11/bin ← cp2k/tools/toolchain/install/elpa-2022.11/bin/cpu/
cp2k/tools/toolchain/install/elpa-2022.11/lib ← cp2k/tools/toolchain/install/elpa-2022.11/cpu/lib
cp2k/tools/toolchain/install/elpa-2022.11/include ← cp2k/tools/toolchain/install/elpa-2022.11/cpu/include

1.2.3.1.2 Other External Libraries Some users have also reported issues (usually when compiling MixPI) that it cannot locate certain external library functions (usually related to either libxc or libsxmm). If this error occurs, it is usually a result of a mismatch between specific versions of libxc/libxsmm and the versions of precompiled functions on the HPC. The user can track down these differences, but in this case, many of the external libraries are not needed to run the relevant MD simulations. Therefore, we suggest adding –with-LIBNAME=no to the toolchain build (similar to the ELPA example above). We recommend the install line:
./install_cp2k_toolchain.sh –with-elpa=no –with-libxc=no –with-libxsmm=no

2 Running MixPI

2.1 Input

In this section, we introduce the specific input files needed to run the MixPI software program. We provide input files for two example simulations, a box of quantized water and a solvated metal M^3+ ion with a PI electron, in the **examples** directory. In the text below, we review the input files and provide detailed explanations on the parameters. While the **example** directory contains input files for the full systems, for ease of demonstration, we will illustrate the input files for two simplified systems with reduced numbers of atoms and RP beads:

- Two water molecules with the hydrogen atoms treated with 4 beads and the oxygen atoms treated with 2 beads
- M^{3+} ion with two classical water molecules and an 8 bead PI electron centered at the ion.

The input files can be classified in three categories:

- MixPI input with PI specific information (such as number of PIs, size of PIs, etc.): ***rp_molecule_info.inp***
- CP2K input files related to the force/energy evaluations and the structure and connectivity of the system: ***cp2k.inp, structure.xyz, structure.psf, rp_molset.dat and classical_molset.dat***
- MixPI input with information on the MD simulations (step size, temperature, etc.) and system specific input files that are only required for particular simulation types: ***md_run.inp, file_info.inp, xx_rcut.inp, md_constraint.inp***

.

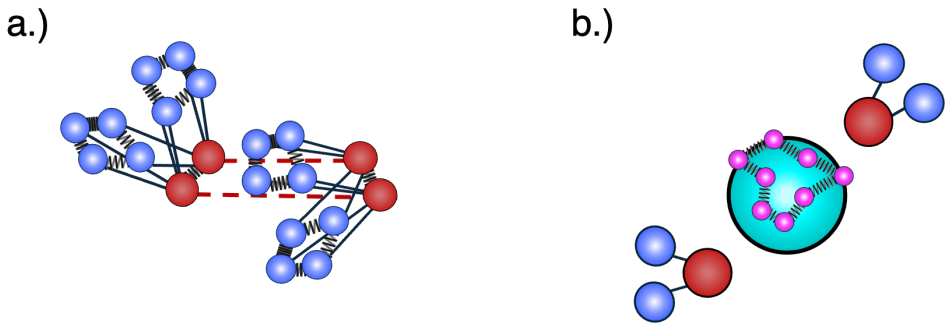


Figure 1: Cartoon representations of the two example systems.

2.1.1 MixPI Input Files for PI Specific Information: *rp_molecule_info.inp*

A file, called ***rp_molecule_info.inp***, contains information on which molecules contain atoms treated as PIs and their respective number of beads. The first line lists the number of molecules that contain at least one PI atom. A block of information is then repeated for each molecule type. The first line of this block contains: number of atoms in the molecule, number of molecules of this type, and the PSF file that describes this molecule type. After this line, there is a line for each atom in the molecule which contains the number of beads for that atom. If specifying a kinetic mass (which is optional and discussed below), you also specify the kinetic mass here.
In Fig. 2, we show the ***rp_molecule_info.inp*** file for the two systems we use for demonstration, respectively: (a) the simulation of 2 water molecules where the O atoms are treated as 2 bead RPs and the H atoms are treated as 4 bead RPs, and (b) the simulation with one M^{3+} ion and two classical water molecules with an 8 bead PI electron centered at the ion. Note that for system (b), the electron and M^{3+} are treated as one molecule where artificial bonds are defined between electron RP beads and M^{3+} (for reasons explained in section 2.1.4.3.1), and the two water molecules are treated as classical and thus not included in this file. This file ***rp_molecule_info.inp*** is used by the input generation program ***make_pimd_input.F*** to convert classical CP2K input files to the RP input files required by MixPI (see section 2.1.2 below). This file is also required to run PI simulations with the MixPI program.

(a)

```
2                          !no. of molecules with RPs
3    2   SPCFw.psf         !no. of atoms in molecule type; no. of molecule type in system; PSF file
     2          8.00       !no. of RP beads atom 1; optional kinetic mass
     4          0.25       !no. of RP beads atom 2; optional kinetic mass
     4          0.25       !no. of RP Beads atom 3; optional kinetic mass
```

(b)

```
1                          !no. of molecules with RPs
2    1   electron.psf      !no. of atoms in molecule type;  no. of molecule types in system; PSF file
     8          0.5        !no. of RP beads atom 1; optional kinetic mass
     1         58.993      !no. of RP beads atom 2; optional kinetic mass
```

Figure 2: The ***rp_molecule_info.inp*** file for (a) a simulation of 2 water molecules described by the ***SPCFw.psf*** file with 2 bead RPs for the oxygen atoms and 4 bead RPs for the hydrogen atoms, and (b) the simulation with one M^{3+} ion and two classical water molecules with an 8 bead PI electron centered at the ion. Note that for system (b), M^{3+} and the electron RPs are treated as one molecule and is described by the ***electron.psf***, while the two water molecules are treated classically and thus not included in this file.

2.1.2 Input Generation Script: *make_pimd_input.F*

The script ***make_pimd_input.F***, located in the **examples/generate_input** directory, will create the MixPI version of the CP2K input files (outlined below) from the corresponding classical input files. To run this program, compile using the following:
gfortran -ffree-form -O2 make_pimd_input.F -o make_pimd_input.out
to create an executable. You will need the following input files:

- ***rp_molecule_info.inp***: an input file that outlines the specific atoms and RP sizes for the system. This file format is also used by MixPI and is described in Sec. 2.1.1.

- ***structure.xyz***: the structure of the classical system with RPs represented by their centroid positions

- ***NAME.psf***: PSF files for the classical systems that contain PI atoms. The file name is up to the user and is specified in the ***rp_molecule_info.inp*** file.

- ***FF_info.inp***: the input file for the classical force-field parameters. This file is currently a list of bond, bend, and Lennard-Jones parameters.

The input generation script will generate MixPi CP2K input files that are needed for a MixPi simulation. These output files are listed as below and are further explained in 2.1.3:

- ***RP_structure.xyz***: the coordinates of the system with all RP beads and classical atoms based on the classical structure given in ***structure.xyz*** and the specific quantization scheme specified in ***rp_molecule_info.inp***. This is explained further in Sec. 2.1.3.2

- ***rp-##-NAME.psf***: the PSF file that describes the topology of each molecule that has atoms quantized as RPs. This file is generated according to the classical connectivity given by ***NAME.psf*** and the quantization scheme specified in ***rp_molecule_info.inp***. The '##' is replaced by a number designation that indexes the number of molecules that contain RPs. The bond and angle sections are based on the classical connectivity described at the classical PSF file and adjusted based on the MixTS rules (see section 2.1.3.1). This file is further explained in Sec. 2.1.3.3.

- ***RP_FF_bond.inp*** and ***RP_FF_nonbond.inp***: CP2K force-field files which describe the bonded and nonbonded interactions between RP beads and RP-classical atoms according to the classical force-field parameters provided in ***FF_info.inp*** (see the examples in **examples/generate_input**) and the quantization scheme specified in ***rp_molecule_info.inp***. See Sec. 2.1.3.1 for further details.

- ***RP_type.pot***: the CP2K file that defines the atom types for the RPs. The harmonic masses of the RP beads are calculated and given in this file according to the masses in ***rp-##-NAME.psf***. Note that the masses defined in this file are the harmonic masses, while the kinetic masses, if different from the harmonic masses, are defined in ***rp_molecule_info.inp***.

- ***RP_molset.pot***: a CP2K file that describes the system composition by defining the number of molecules for each type of PSF file. See Sec. 2.1.3.3 for further examples and explanations.

Examples of the input files needed for ***make_pimd_input.F*** and the resulting output files are shown in the **examples** directory. In section 2.1.3, we outline the format for the output files that are generated. This can be used if the user wishes to construct their own MixPI CP2K input files which may be necessary if the force-field type is beyond the currently implemented formats in ***make_pimd_input.F***. For example, ***make_pimd_input.F*** does not currently support the following: dihedrals, harmonic constraints on atoms or RPs, definitions of proper collective variables, or extneral potentials that utilize the GEN_POT functionality. However, all of these situations can be handled by MixPi if the user edits the CP2K input files as described by the relevant sections below.

2.1.3 CP2K Input Files

A set of CP2K input files, similar to those needed for a traditional CP2K MD simulation, are required. While the inputs are structured like classical CP2K MD input, the MixPI CP2K input files need to be modified to reflect the accurate force and structure information for the path-integral set-ups. *A simple program (called **make_pimd_input.F** which is located within the **examples/generate-input** directory) can take the classical CP2K input files and the PI specific information (found in **rp_molecule_info.inp**) and convert them into MixPI CP2K inputs*; more information on this program is provided in Sec. 2.1.2. Each of the CP2K input files is outlined in a section below.

The CP2K input file (called ***cp2k.inp*** in our **example** directory) contains the input parameters dedicated to the force evaluations and system structure. (In the CP2K input file, the section FORCE_EVAL contains the subsection MM to describe the forces and the subsection SUBSYS to describe the coordinates and topology of the system. Additionally, subsections like CONSTRAINT and COLVAR, which

affect the underlying force evaluations, are also read in by the MixPI implementation of CP2K.) On the other hand, any parameters in the CP2K input concerning molecular dynamics parameters (like temperature, time step, etc.) are ignored. This section will outline the information needed to accurately describe the force evaluations in the FORCE_EVAL section, how to scale the force evaluations for MixTS configurations, and how to describe the topology in the SUBSYS section of a CP2K input file. A basic CP2K input file is provided in the **examples/** directory that reads in the force definitions generated by ***make_pimd_input.F***. However, if the user is planning to use force definitions beyond those currently implemented in ***make_pimd_input.F***, changes to cp2k.inp will need to be made by the user.

The implementation describe below uses the internal CP2K format to describe the forces, along with the MOL_SET functionality combined with PSF files. In the future, implementations using other formats in CP2K (like CHARMM and AMBER formats) will be added.

2.1.3.1 Force Information in FORCE_EVAL: *cp2k.inp* The force definitions used in ***cp2k.inp*** must describe all the interactions in the system including the interactions between the classical atoms and RP beads, the RP beads on separate atoms, and the classical atom interactions. Given the classical forcefield parameters, the correct force definitions for RP-RP and RP-atom interactions can be automatically generated using ***make_pimd_input.F*** for a PI simulation (see Sec. 2.1.2).

If the user chooses to generate their own input files and force definitions, these interactions are defined by the MixTS Hamiltonian;

$$H_{\mathrm{RP}}^{\mathrm{mix,2\text{-}body}} = H_0 + V_{\mathrm{mix}} \tag{1}$$

$$H_0 = \sum_{i=1}^{N_{\mathrm{atom}}} \sum_{\alpha=1}^{N_i} \frac{p_{i,\alpha}^2}{2m_i} + \frac{N_i m_i}{2\beta^2\hbar^2}(q_{i,\alpha} - q_{i,\alpha+1})^2 \tag{2}$$

$$\begin{aligned} V_{\mathrm{mix}} = & \sum_{i=1}^{N_{\mathrm{atom}}} \frac{1}{N_i} \left(\sum_{\alpha=1}^{N_i} V_i(q_{i,\alpha}) + \right. \\ & \left. \sum_{j>i} \sum_{\gamma=1}^{\frac{N_i}{N_j}} \left(\sum_{\alpha=1}^{N_j} V_{ij}(q_{i,(\alpha-1)\frac{N_i}{N_j}+\gamma}, q_{j,\alpha}) \right) \right), \end{aligned} \tag{3}$$

where $H_{\mathrm{RP}}^{\mathrm{mix,2\text{-}body}}$ is the MixTS Hamiltonian with forces and energies truncated to only 2-body interactions, H_0 is the free RP Hamiltonian, and V_{mix} contains the forces between different atoms and RPs in the system. N_{atom} is the number of atoms in the simulation and N_i is the number of beads for the ith atom; we assume that the particles are numbered so that $N_1 \geq N_2 \geq ... \geq N_{N_{\mathrm{atom}}}$. Additionally, $q_{i,\alpha}$ and $p_{i,\alpha}$ are the position and momentum for the ith atom and α bead, m_i is the mass of the ith atom, $\beta = 1/k_bT$ where k_b is the Boltzmann constant, T is temperature, and $\hbar$ is Planck's constant. The potential terms are separated into $V_i(q_{i,\alpha})$, the one-body potential for atom i and bead α, and $V_{ij}(q_{i,\alpha}, q_{j,\beta})$, the two-body interaction between bead α on atom i and bead β on atom j. As noted in Eq. 3, there are selection rules for which two bead numbers (α and β) will interact with each others. These rules, as well as the derivation of the above Hamiltonian, are explained in Section 4.3.

Here we use the system with two water molecules using a qSPCF/w water model to explain how the forcefield parameters are calculated for a PI simulation according to the MixTS Hamiltonian shown above. In a classical version of this system, we have electrostatic interactions between atoms in the two water molecules, Lennard-Jones interactions between the O atoms, harmonic bond interactions between the H and O atoms, and harmonic angle interactions between two O-H bonds. In our example where the oxygen atoms are represented with a 2-bead RPs and hydrogen atoms with 4-bead RPs, the forces are calculated between the RP beads as shown in Fig. 1a according to the MixTS Hamiltonian.

2.1.3.1.1 Lennard-Jones Interactions We are using a Lennard Jones potential

$$V_{\mathrm{LJ}} = 4\epsilon_{OO}\left[\left(\frac{\sigma_{OO}}{r_{OO}}\right)^{12} - \left(\frac{\sigma_{OO}}{r_{OO}}\right)^{6}\right] \tag{4}$$

to describe the non-bonded interactions between the two classical oxygen atoms. A Lennard-Jones block for ***cp2k.inp*** is shown in Fig. 3a to describe the interaction between two classical oxygen atoms (ϵ_{OO} = 79.2654 K, σ_{OO} = 3.165492 $\dot{A}$). The interaction strength needs to be modified to account for

```
a)  &LENNARD-JONES
                        ATOMS                OW    OW
                        EPSILON [K_e]                78.27
                        SIGMA [angstrom]             3.17
                        RCUT [angstrom]              7.00
    &END LENNARD-JONES
b)  &LENNARD-JONES
                        ATOMS 40W            4OW   4OW
                        EPSILON [K_e]                39.13
                        SIGMA [angstrom]             3.17
                        RCUT [angstrom]              7.00
    &END LENNARD-JONES
```

Figure 3: The Lennard Jones parameters between two oxygen atoms in a (a) classical simulation and (b) PI simulation where the oxygen atoms are treated as 2 bead RPs using the q-SPC/Fw force-field. For the PI simulation, the force is scaled by the number of PI beads (in this case, 2); therefore, the ϵ parameter in (b) is half the value of the classical epsilon parameter in (a). The σ parameter does not change. The exact scaling for these interactions is explained by the two body V_{ij} term in Eq. 3.

```
a)  &BOND
                        ATOMS                OW      HW
                        K [angstrom^-2kcalmol] 1059.16
                        R0 [angstrom]          1.00
                        KIND G87
    &END BOND
b)  &BOND
                        ATOMS                2OW     4HW
                        K [angstrom^-2kcalmol] 264.79
                        R0 [angstrom]          1.00
                        KIND G87
    &END BOND
```

Figure 4: The bonded parameters for an O-H bond in a (a) classical simulation and (b) PI simulation where the oxygen atoms are treated as 2 bead RPs and the hydrogen atoms are treated as 4 bead RPs using the q-SPC/Fw force-field. For the PI simulation, the force is scaled by the number of PI beads for the most quantized atom. (In this case, $N_H = 4; N_O = 2$ so the bonded interaction is scaled by $\frac{1}{4}$). Therefore, the spring constant k in (b) is $\frac{1}{4}$ the value of the classical spring constant in (a). The r_0 parameter does not change. The exact scaling for these interactions is explained by the two body V_{ij} term in Eq. 3.

replacing the classical oxygen atoms with 2-bead RPs. Following the MixTS Hamiltonian shown in Eq. 3, we need to scale the strength of the interaction by $\frac{1}{N_\alpha} = \frac{1}{2}$. In Fig. 3b, we show the Lennard-Jones block to describe the interaction between two 2-bead RP oxygen atoms ($\epsilon_{OO} = 39.1327$ K; $\sigma_{OO} = 3.165494$ $\dot{A}$). In Fig. 1a, the Lennard-Jones interactions between the 2-bead RPs are shown by the dashed red lines.

2.1.3.1.2 Bonded Interactions A similar methodology is used to describe the harmonic bond interactions between the H and O atoms and the harmonic angle interactions between the O-H bonds within any water molecule. For example, Figure 4a shows the CP2K input block to describe the harmonic bond between classical H and O atoms in a water molecule ($k = 1059.162$ kcal/mol$\dot{A}$, $r_0 = 1.00$ $\dot{A}$). We need to scale this interaction to account for treating the hydrogen atoms as 4-bead RPs and the oxygen atoms as 2-bead RPs. Based on the form for V_{mix} in Eq. 3, we need to scale the interaction by $\frac{1}{4}$; the description for the bond between the 4-bead RP H and the 2-bead RP O is shown in Fig. 4b ($k = 264.7905$ kcal/mol$\dot{A}$, $r_0 = 1.00$ $\dot{A}$). As mentioned above, specific rules, shown in the Eq. 3, apply for which pair of beads will have a bonded interaction. In this case, the bonded interactions are shown as solid blue lines in Fig. 1.

2.1.3.1.3 Electrostatic Interactions The electrostatic interactions have a different implementation than the other nonbonded interactions and the intramolecular interactions in MixPI. The pairwise Coulomb interaction between two atoms can be written as

$$V_{\text{electrostatic}} = \frac{q_i q_j}{|r_{ij}|} \tag{5}$$

where q_i is the charge of the ith particle and $|r_{ij}|$ is the distance between the ith and j^{th} particle. If we cast this interaction into the mixTS framework from Eq. 3, the Coulomb interaction between two particles should be

$$V_{\text{electrostatic}} = \frac{1}{N_i}\sum_{\alpha=1}^{N_j}\left(\sum_{\lambda=1}^{\frac{N_i}{N_j}}\left(\frac{q_{i,(\alpha-1)\frac{N_i}{N_j}+\lambda} q_{j,\alpha}}{|r_{(i,(\alpha-1)\frac{N_i}{N_j}+\lambda)(j,\alpha)}|}\right)\right) \tag{6}$$

where the ith atom is quantized with N_i particles and $N_i > N_j$. This expression follows the bead-bead interactions defined in the mixTS Hamiltonian. If a RP is interacting with a classical (1 bead) atom ($N_j = 1$), then you obtain the correct scaling of the potential by bringing the $1/N_i$ term inside the sum and dividing the charge on the RP beads by N_i ($q_{(i,(\alpha-1)\frac{N_i}{N_j}+\lambda)} = \frac{q_{(i,\lambda)}}{N_i}$). Therefore, in a simulation in which one particle is treated as a RP (like the metal cation + RP electron simulation), the electrostatic interactions are treated in the true mixTS format.

Unlike the Lennard-Jones interactions, there is no way to define the Columbic parameters to have a consistent $1/N_i$ scaling for the interaction between RPs. If we scale all the charges by the number of beads on that RP, we have an extra $1/N$ for each pairwise interaction. Therefore, we choose to utilize a "generalized" PIMD Hamiltonian for the Coulomb interactions. In this approach, which is derived in the Appendix of the original paper, the mixTS Hamiltonian for two particles can be exactly written as

$$H_{\text{RP}}^{\text{gen}} = \sum_{i=1}^{2}\left(\sum_{\alpha=1}^{N_i}\frac{p_{i,\alpha}^2}{2m_i} + \frac{N_i m_i}{2\beta^2\hbar^2}(q_{i,\alpha} - q_{i,\alpha+1})^2\right) + \frac{1}{N_1}\sum_{\alpha=1}^{N_1}\frac{1}{N_2}\sum_{\gamma=1}^{N_2} V(q_{1,\alpha}, q_{2,\gamma}) \tag{7}$$

in the limit of $N \to \infty$. We use the above potential definition with the electrostatics by including the $1/N$ scaling by dividing the charge on every RP bead by the total number of beads on that particle while allowing all beads on particle 1 to see all beads on particle 2.

The general expression for the approximation is

$$\frac{1}{N_i}\sum_{\alpha=1}^{N_j}\left(\sum_{\gamma=1}^{\frac{N_i}{N_j}}\left(V_{ij}(r_{i,(\alpha-1)\frac{N_i}{N_j}+\gamma}, r_{j,\alpha})\right)\right) \approx \sum_{\alpha=1}^{N_j}\left(\sum_{\gamma=1}^{N_i}\frac{1}{N_i}\frac{1}{N_j}V_{ij}(r_{i,\gamma}, r_{j,\alpha})\right) \tag{8}$$

. In Fig. 5, we show a diagram describing the interactions between the 4-bead hydrogen atom and the 2-bead oxygen atom on neighboring water molecules using both the mixTS Hamiltonian and the generalized treatment used in MixPI. For the water example (2 water molecules with 4-bead H and 2-bead O atoms), the Coulombic interaction between bead 1 on the 4-bead H with the 2-bd oxygen in the neighboring water molecule uses the following approximation:

$$\frac{\frac{q_{H,1}}{4} q_{O,1}}{(r_{H,1} - r_{O,1})} \approx \sum_{\alpha=1}^{2}\frac{\frac{q_{H,1}}{4}\frac{q_{O,\alpha}}{2}}{(r_{H,1} - r_{O,\alpha})} \tag{9}$$

.

To show the effect of this generalized approximation, we compare the electrostatic potential using the true mixTS treatment, the generalized treatment, and the centroid approximation for the mixTS N=(32,4) water simulations in Fig. 6. The generalized potential (blue squares) is very similar across the entire trajectory to the true mixTS treatment (green circles). To show that the two treatments will generate the same distributions, Fig. 7 tracks the potential energy over a 15 ps trajectory for the mixTS N=(32,4) water simulations where the LJ interactions are treated with the mixTS Hamiltonian (orange) and the generalized Hamiltonian (blue). The electrostatics are treated with the generalized Hamiltonian in both. In 8, we compare the average energies from these two trajectories. We see that while the trajectories traces are different, the average properties are the same for both treatments.

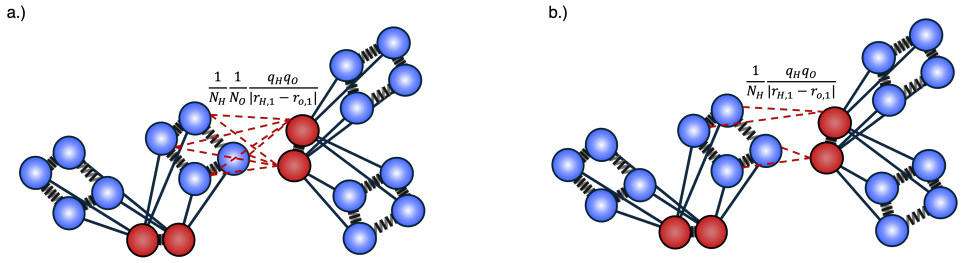


Figure 5: A comparison of the electrostatic interactions between a 4-bead H and a 2-bead oxygen atoms on neighboring water molecules via a) the generalized Hamiltonian used for electrostatics in MixPI and b) the MixPI Hamiltonian used for other interactions in MixPI.

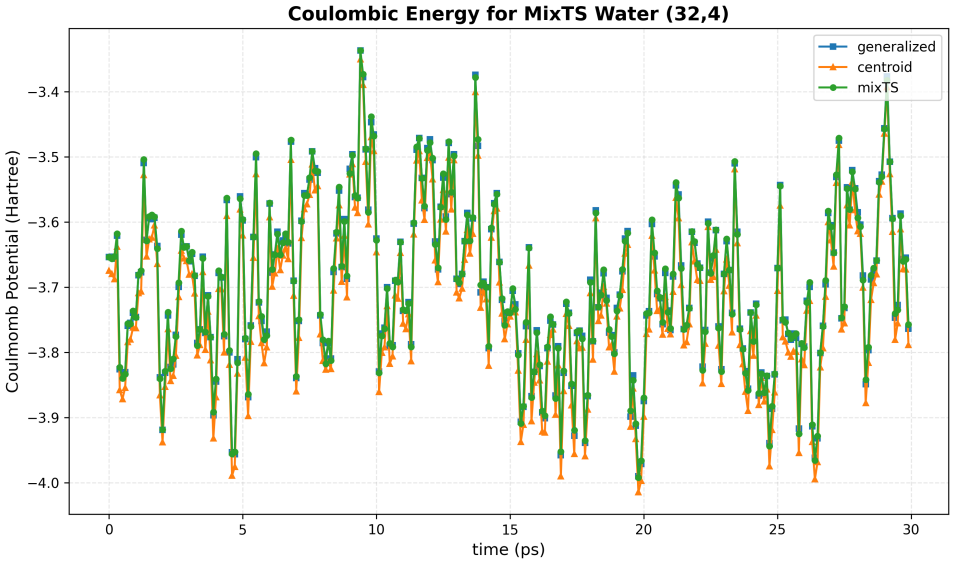


Figure 6: The electrostatic potential along a single trajectory taken from a mixTS $N = (32, 4)$ water simulation generated with MixPI is shown here. For each snapshot of the system, we compute the electrostatic potential from the mixTS expression (green circles), the generalized potential used in MixPI (blue squares), and the centroid approximation (orange triangles). As expected, we find the generalized and MixTS potentials are identical while the centroid approximation to the potential is less accurate.

2.1.3.2 Coordinate Information: *structure.xyz* A coordinate file that is readable by CP2K (file types include xyz, pdb, g96, crd, and cif) lists the positions of all the PI beads and classical atoms in the system. This file is specified by the keyword COORD_FILE_NAME in ***cp2k.inp***. Note, the ordering of the molecules in this file is important. The atoms are listed by molecule, and the molecules that contain atoms treated as PIs should be listed before any classical molecules. In Fig. 9a, we show an XYZ coordinate file for a water dimer system with 2 beads on the oxygen atoms and 4 beads for the hydrogen atoms (shown as a cartoon in Fig. 1a). In Fig. 9b, we show an XYZ file for a classical M^{3+} ion with a PI electron (treated as 8 beads) and classical water (shown as a cartoon in Fig. 1b). In this particular example, the M^{3+} ion is bonded to the electron (since the electron should be centered on the M^{3+} ion for this simulation) with a harmonic spring constant of 0. Because of this bond, the M^{3+}-electron complex is treated as a molecule. The M^{3+}-electron "molecule" is listed first in the XYZ file (since this molecule contains a PI) and the classical waters are listed last. Also note that the atom label used in column 1 does not matter; the user can choose whichever atom labels are convenient. Here we have chosen to designate the metal cation as "Co."

2.1.3.3 Topology Information: *structure.psf; rp_molset.dat; and classical_molset.dat* Inside the TOPOLOGY section of the ***cp2k.inp*** file there are various methods for specifying how atoms are bonded to one another. Here we have chosen to use the method which uses PSF files and the MOL_SET section of CP2K. Below, we will outline how these files work together. Other methods for specifying bonding are allowed but the naming conventions outlined below must be followed.

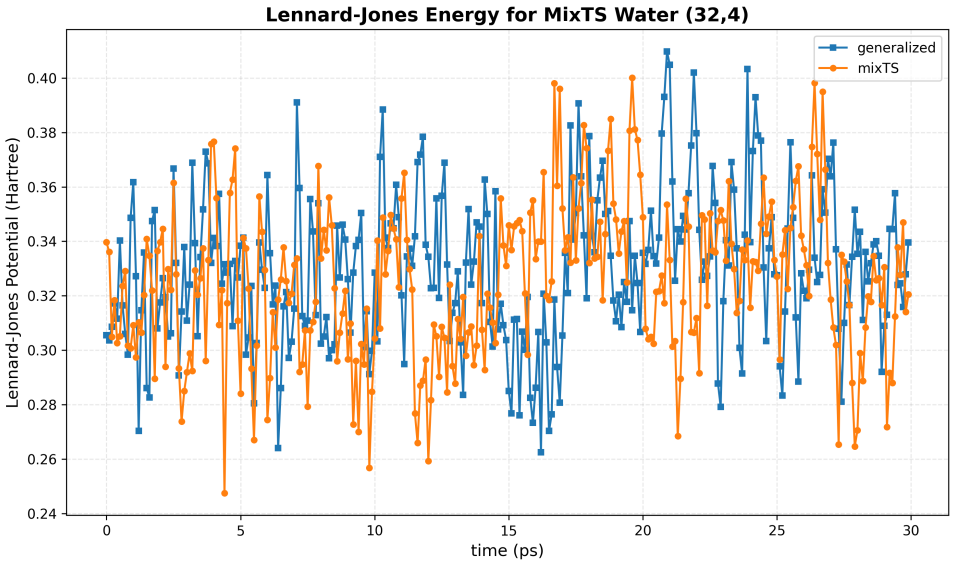


Figure 7: Comparing the Lennard-Jones potential energy along two different 15 ps mixTS trajectory of N=(32,4) generated by treating the electrostatics with a generalized potential and the Lennard-Jones interactions using either the generalized form (blue) or the mixTS form (orange). In this test, because the trajectories are generated using two different potentials, the generalized and mixTS LJ energies are not exactly the same, but are comparable.

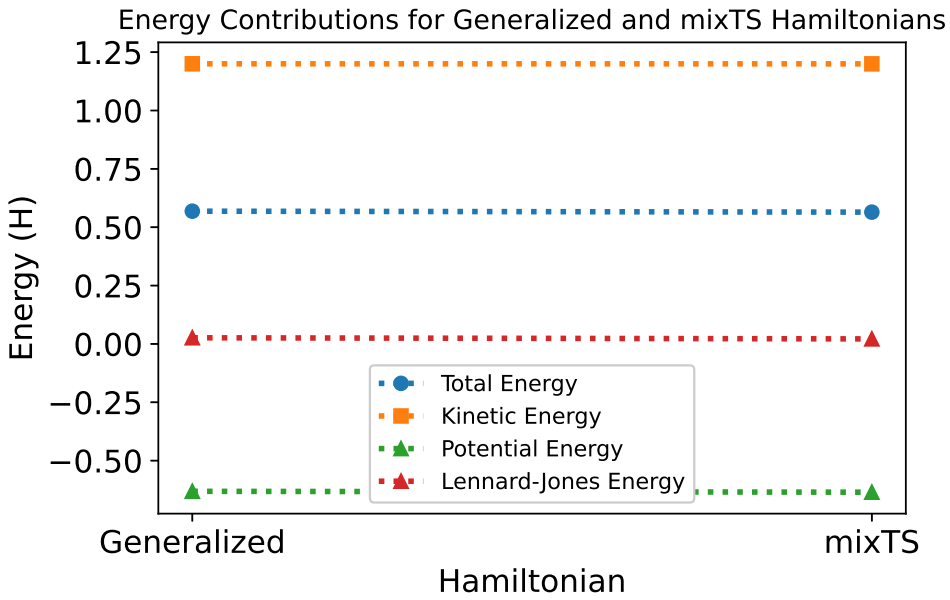


Figure 8: Comparing the average total, kinetic, potential, and Lennard-Jones energies over two 5 ps mixTS N=(32,4) trajectories in which the electrostatics are treated with the generalized potential form and the Lennard-Jones potential is treated with either the generalized form (left) and the mixTS form (right). While individual trajectories generated from the two different potentials will not be identical (as shown earlier), the ensemble average is identical.

2.1.3.3.1 PSF File A PSF file (protein structure file) is used to indicate the atom types and connectivity information for the system. A PSF file for "classical" q-SPC/Fw water and PI q-SPC/Fw water is shown in Fig. 10a and b, respectively. The first block contains the atomic information. The order of the columns is: atom ID, segment name, residue ID, residue name, atom name, atom type, charge, mass, and an unused 0. As we will see below, the important columns are segment name, atom name, atom type (which, when you compare to Fig. 3, matches the force-field information), charge, and

a) 20

OW	1.5211919	3.5998747	-0.652675
OW	1.4121281	3.5802349	-0.661211
HW	1.4005127	4.4875559	-1.271614
HW	1.5174949	4.3678642	-1.348312
HW	1.5644747	4.3707807	-1.380281
HW	1.5408096	4.3464767	-1.201648
HW	2.4998152	3.6217743	-0.620695
HW	2.4262301	3.7494224	-0.572054
HW	2.3040601	3.8964222	-0.492431
HW	2.3936183	3.9456111	-0.407478
OW	7.1441669	5.9479881	1.9757871
OW	7.1055023	5.9521904	1.9650458
HW	7.9928307	6.2423251	2.3580867
HW	8.0249263	6.3070954	2.3374645
HW	8.0612774	6.1868817	2.2635323
HW	8.0762995	6.1007724	2.2638971
HW	7.3148659	5.6370625	1.0306234
HW	7.2993247	5.5476543	1.0564134
HW	7.2855243	5.4725721	1.0394881
HW	7.2861469	5.4444205	1.0113705

b) 15

XX1	7.4998214	7.0600361	7.4657116
XX2	7.3750055	6.9603192	7.2230723
XX3	7.0379188	6.7793014	7.1562945
XX4	7.1721635	6.8855996	7.0493341
XX5	7.4513039	6.9700067	7.205495
XX6	7.3320077	6.7846779	7.3081813
XX7	7.3242461	6.9381174	7.4106001
XX8	7.2016851	6.8800609	7.4670236
FE1	7.2514378	7.0334794	7.2374937
O	8.3625722	10.114932	8.2990075
H	7.513133	10.679729	8.3130297
H	9.1916837	10.69787	8.5837361
O	3.6041147	9.5224198	10.496797
H	2.940447	9.6895883	11.342763
H	3.04389	9.6005504	9.6307834

Figure 9: Two example configuration files (in the xyz format) for (a) a qunatized water simulation of two water molecules with 2 beads on the oxygen atoms and 4 beads on the hydrogen atoms and (b) a simulation of a classical M^{3+} with an 8 bead PI electron solvated with two classical water molecules. These two configuration files are simpler examples of the configurations used in the example calculations. Note that in the case of the M^{3+} simulation, the molecules that contain PI are listed before the classical water molecules. We have chosen an atom label of "Co" for the metal cation; this choice was arbitrary.

mass. The format required for the PI variables in the atom definition block is as follows:

- Column 1: atom number. All particles (atoms or PI beads) are numbered starting at 1. There is no change from the classical treatment.
- Column 2: molecule name. This name must start with "RP" if the molecule contains any ring polymers followed by the total number of particles (total RP beads+total number atoms in molecule) in the molecule. In this example, we used RP10 as a molecule name (since it does contain RPs, and

has 10 total beads in the molecule: 2 RP beads on the oxygen, 4 RP beads on the first hydrogen, 4 RP beads on the second hydrogen).

- Column 3: molecule number. This can be any number. No change from the classical treatment.
- Column 4: atom type. The atom type starts with a two character atom identifier (in this case, an atomic kinds descriptor like "HW" but can be whatever the user chooses) and the bead number of that atom within its ring polymer.
- Column 5: force type. This is the force-field descriptor for that atom type. Since the force is dependent on atom type and the number of beads for the RP, the convention is the number of beads on that RP (N_{atom}) and a two-character atom-type descriptor (which does not need to match the one used in Column 4, we choose to do so in this case).
- Column 6: charge. This is the charge of the atom divided by the number of beads.
- Column 7: mass. This is the mass of the atom divided by the number of beads.
- Column 8: a dummy variable.

The second, third, and fourth sections of the PSF file defines the bonds, bends, and dihedral angles relative to the atom numbers specified in column 1 above. These definitions are consistent with the classical descriptions when combined with the force matching selection rules between atoms and beads as defined in Eq. 3.

2.1.3.3.2 MOL_SET Information In Fig. 11, we show an example of the input "TOPOLOGY" section of ***cp2k.inp*** for the Co+PI electron with 2 water molecules (the configuration file is shown in Fig. 9b). The MOL_SET section allows a molecular PSF file to be repeated a specified number of times in a system. In this example, we have one PSF file that contains information about the M^{3+} + PI electron "molecule". We have a second PSF file that describes the connectivity of a single water molecule and is repeated twice (NMOL 2) in order to describe the two water molecules in the system. For this method, we only need to create one PSF file for each unique molecule type in our system.

2.1.4 MixPi Input Files for Molecular Dynamics Inputs

2.1.4.1 ***md_run.inp*** This file contains the information about the molecular dynamics methods. In order, the following variables are specified.

- nsteps (int): number of molecular dynamics steps
- print_freq (int): frequency to save and print and postions and energies
- dt (real): size of time step in fs
- method (string): pimd (specify kinetic mass in ***rp_molecule_info.inp***) or rpmd (uses atomic mass/N_i)
- ensemble (string): nve or nvt
- thermostat (string): if ensemble = nvt, specify thermostat as pile (path-integral Langevin equation) or pileg (path-integral Langevin equation global where only the centroid velocities are scaled). If ensemble=nve, do not include this line.
- tau_classical (real): if ensemble = nvt, classical thermostat variable τ_0. Will only be used if purely classical atoms are present.
- tau_PI (real): if ensemble = nvt, PI thermostat variable τ_0
- temperature (real): temperature to run simulation and initialize velocities
- propagation (string): method for propagation of PI normal modes (none=manually evaluate springs and propagate with velocity-verlet; normal_manual=propagate using normal mode propagation).
- init_vel (logical): initial velocities to Boltzmann distribution or read in from file (***rp-vel.dat***)

a) PSF

```
1 !SPC/Fw

3 !NATOM
1 MOL1 1 SOL OW OW -0.84 15.998 0
2 MOL1 1 SOL HW HW 0.42 1.008 0
3 MOL1 1 SOL HW HW 0.42 1.008 0

2 !NBOND
1 2 1 3

1 !NTHETA
2 1 3

0 !NPHI

0 !NIMPHI
```

b) PSF

```
1 !SPC/Fw with PI

10 !NATOM
1 RP10 1 RP OW1 2OW -0.42 7.9995 0
2 RP10 1 RP OW2 2OW -0.42 7.9995 0
3 RP10 1 RP HW1 4HW 0.105 0.252 0
4 RP10 1 RP HW2 4HW 0.105 0.252 0
5 RP10 1 RP HW3 4HW 0.105 0.252 0
6 RP10 1 RP HW4 4HW 0.105 0.252 0
7 RP10 1 RP HW1 4HW 0.105 0.252 0
8 RP10 1 RP HW2 4HW 0.105 0.252 0
9 RP10 1 RP HW3 4HW 0.105 0.252 0
10 RP10 1 RP HW4 4HW 0.105 0.252 0

8 !NBOND
1 3 1 4 2 5 2 6
1 7 1 8 2 9 2 10

4 !NTHETA
3 1 7 4 1 8 5 2 9
6 2 10

0 !NPHI

0 !NIMPHI
```

Figure 10: The PSF file for (a) classical and (b) PI water (2-bead oxygen atom and 4-bead hydrogen atoms) q-SPC/Fw water molecule.

```
&TOPOLOGY
 COORD_FILE_NAME      RP_structure.xyz
 COORD_FILE_FORMAT    XYZ
 CONN_FILE_FORMAT     MOL_SET
 &MOL_ SET
  &MOLECULE
   NMOL                1
   CONNECTIVITY        PSF
   CONN_FILE_NAME      rp-electron-co.psf
  &END MOLECULE
  &MOLECULE
   NMOL                2
   CONNECTIVITY        PSF
   CONN_FILE_NAME      water.psf
  &END MOLECULE
 &END MOL_SET
&END TOPOLOGY
```

Figure 11: The TOPOLOGY section of the ***cp2k.inp*** file that uses the MOL_SET parameters to specify the connectivity for the Co+PI electron with 2 water molecules.

- constraints (logical): are constraints present (frozen atoms; constrained centroids-see file ***md_constraint.inp***)
- print_level (string): low/high. Output files for each level are described in Sec. 2.2.

Figure 12 shows an example ***md_run.inp*** file.

```
200                 !number of MD steps
1                   !frequency to save/print positions, energies, etc.
0.5                 !timestep in fs
pimd                !method (pimd or rpmd)
nvt                 !ensemble (nve or nvt)
pileg               Thermostat (pile or pileg)
200                 !thermostat time constant classical atoms
200                 !thermostat time constant PI beads
300                 !temperature
normal_manual       !propagation (none, normal_manual)
.true.              !initialize velocities (if false, read in from file)
.false.             !constraints present (frozen atoms, centroids)
low                 !print level (low, high)
```

Figure 12: The ***md_run.inp*** file for a simulation of an example simulation.

2.1.4.2 ***file_info.inp*** This file outlines information about the CP2K input files. The format is as follows:

- cp2k_input (string): name of CP2K input file
- cp2k_output (string): name of output file for CP2K print statements and error messages
- structure_file (string): name of initial structure file (usually xyz). Should match the structure specified in the CP2K input file.
- classical_atoms (logical): Indicates if purely classical molecules exist.
- molset_file (string): If the above logical is true, this line is read in. It specifies name of MOL_SET file that describes the classical molecules. See Sec. 2.1.3.3.2 for information on this file format.

```
'cp2k.inp'              ! CP2K input file
'output.out'            ! CP2K output file
'RP_structure.xyz'      ! initial structure file
.true.                  ! classical atoms present
classical_molset.pot'   ! CP2K mol_set file if classical atoms present
```

Figure 13: The ***file_info.inp*** file for an example simulation.

An example ***file_info.inp*** file is shown in Fig. 13 The classical molset file, which is specified in the final line of the ***file_info.inp*** file when classical atoms are present, is a CP2K input file that describes the structure of a system using multiple copies of the same PSF file. See the relevant section of the CP2K manual for additional information. Additional information will also be given in section 2.1.3

2.1.4.3 System-Specific Input Files: *xx_rcut.inp*, *md_constraint.inp* Below we will outline specific situations in which additional input files are required.

2.1.4.3.1 Systems with PI Electrons: Changes to *cp2k.inp* and *xx_rcut.inp* For the simulation of RP electrons, we need to introduce particles that are not traditional atoms into the simulation. As seen in Fig. 9 and Fig. 10, we have chosen the atom name 'XX' to designate an electron. Similar to previous studies [4], we define the electrostatic interaction between an electron bead and all other atoms in our simulation as

$$V(r_{(xx,\alpha),(i,\beta)}) = \begin{cases} \frac{1}{N_{xx}} \frac{q_{i,\beta}}{r_{(xx,\alpha)(i,\beta)}}, & q_{i,\beta} < 0 \\ \frac{1}{N_{xx}} \frac{-q_{i,\beta}}{r_{(xx,\alpha)(i,\beta)}}, & q_{i,\beta} > 0 \text{ and } r_{(xx,\alpha)(i,\beta)} > r_{\text{cut}} \\ \frac{1}{N_{xx}} \frac{-q_{i,\beta}}{r_{\text{cut}}}, & q_{i,\beta} > 0 \text{ and } r_{(xx,\alpha)(i,\beta)} \leq r_{\text{cut}} \end{cases} \tag{10}$$

where N_{xx} is the number of RP beads for the electron and $r_{(xx,\alpha)(i,\beta)}$ is the distance between the α bead of the electron and the β bead of the ith particle. Additionally, $c_{i,\beta}$ is the charge on the β bead of the ith particle. The two bead numbers α and β also follow the selection rules outlined in Sec. 2.1.3.1 ($\beta = (\alpha - 1)\frac{N_{xx}}{N_i} + \gamma$ where N_i is the number of beads for the ith atom and $1 \leq \gamma \leq \frac{N_{xx}}{N_i}$). Equation 10 is a standard Coulombic interaction and applies if the particles have the same charge; however, if the particles are oppositely charged, the interaction is a Coulombic interaction at long distances and is a flat potential when the particles are closer than a cut-off value r_{cut}. Since there are no repulsive interactions between the electron RP bead and the positively charged particles, this flat-potential prevents the beads from localizing at the center of positively charged atom positions. The file xx_rcut.inp is used to introduce this modified electrostatic interaction. This interaction can be introduced to the simulation in two ways both of which are shown by the example xx_rcut.inp file in Fig. 14. The first line lists the number of electron-atom pair (N_{pair}) definitions that are to be modified. The remaining N_{pair} lines list the atom type label (that matches the definition in the PSF file), the value of r_{cut} for the interaction in Eq. 10 and a TRUE/FALSE tag. If false, MixPi will assume that no electrostatic interaction for that pair was included in the forces obtained from CP2K; if true, MixPi will assume a traditional electrostatic interaction was included in the force definition. In the case for the M^{3+} + PI e^- simulation, we used both options. In order to limit the number of force "cancellations," no electrostatic interactions where included in the CP2K calculation between the electron beads and the metal (called FE in this case); however, the CP2K force evaluation did included electron-hydrogen electrostatic interactions. If the the user is careful to not double count any electrostatic interactions, the calculcation is independent on whether the force is included in the CP2K force evaluation or not. In general, we suggest using a definition that will involve the least-number of zeroed-out interactions.

2.1.4.3.2 Systems with Constraints: *md_constraint.inp* If your system contains constraints (including frozen atoms/PI beads, frozen centroids), the input variable for constraints in ***md_run.inp*** is .true. and a secondary file, ***md_constraint.inp*** is used to specify which degrees of freedom are constrained. The following outlines the variables in ***md_constraint.inp***:

- classical_constraints: a logical specifying if any classical atom positions are frozen
  If .true. than the following block is included:

```
        2
FE              1.1     FALSE
 HW               1     TRUE
```

Figure 14: An example of the xx_rcut.inp file for the M^{3+} + PI e^- system.

number of atoms frozen
the atom number for the first frozen atom (from 1 to N where N is the total number of atoms in the system) and the directions for which the atom is frozen (options are x, y, z, xy, xz, yz, xyz)
the atom number for the second frozen atom and the direction it is frozen
...

- pi_constraints: a logical specifying if any centroid atoms are frozen
  .true. than the following block is included:
  string "list" or "range" which specifies that you will provide either a list of atoms with frozen centroids or a range of atoms with frozen centroids. number of PIs for which the centroid is frozen ($N_{centroids}$)
  if *r*ange: the atom number for the first PI with frozen centroid (from 1 to N where N is the total number of PI atoms in the system) and the directions for which the centroids are constrained (options are x, y, z, xy, xz, yz, xyz). It is assumed that the following $N_{centroids}$ atoms are also have frozen centroids.
  if *l*ist: the atom number for the first PI with constrained centroid and the direction it is constrained
  the atom number of the second PI (if $N_{centroid} > 1$) and the direction it is constrained
  ...
  the atom number of the $N_{centroid}$'s atom with a frozen centroid and the direction is it constrained.

- virtual: a logical specifying if any virtual atoms (like those needed for off-site charges) are present

In Fig. 15a, we show an example of the ***md_constraint.inp*** file that would be used for the M^{3+}+ PI e^- simulation in which the M^{3+} ion is treated as 1 bead PI (see Sec. 2.1.3.1 for discussion on why we choose this treatment) and is frozen in the center of the box. In Fig. 15b, we show a similar file (using the range parameter) that would freeze the centroids for the M^{3+} ion and the two water molecules in the simulation (for a total of 7 atoms with frozen centroids).

2.2 Output Files

MixPI generates the following types of output files:

- energy files
- trajectory files
- final velocity and structure files (which can be used to generate a RESTART)
- other files
  - run time file
  - CP2K generated output file

We now describe the structure and content of these files in detail.

2.2.1 Energy Files

Energy files print out system energies (in atomic units) at a printing frequency specified by the user. It includes ***energy_virial.dat***, ***energy_average.dat***, and ***energy_actual.dat***. All energy files are printed in Hartree (E_h).

a)
```
.false.          !frozen atoms
.true.           !Frozen PI centroids
 list            !List/range
 1               !Number of centroids to constrain
 2       xyz     !Atom number with constraint, direction of constraint
.false
```

b)
```
.false.          !frozen atoms
.true.           !Frozen PI centroids
 range           !List/range
 7               !Number of centroids
 2       xyz     !Atom number starts constraint range; direction of constraint
.false
```

Figure 15: The ***md_constraint.inp*** file for the M^{3+}+ PI e^- simulation. In a), the file uses the list parameter to constrain the centroid of the metal center (in this case, the metal is a 1-bead RP and, therefore, this will constrain the actual position). In b), the file uses the range parameter to constrain 7 centroid positions, starting with the centroid of atom 2 through 8. This could be used to freeze the metal center and the two water molecules in the simulation.

- ***energy_virial.dat***: The columns, from left to right, are: time step number, time (in femtosecond), instant_temp (in Kelvin), virial kinetic energy, virial potential energy, primitive kinetic energy, and total energy (virial kinetic plus virial potential energy), respectively.
- ***energy_average.dat***: A running average for the values of the items printed in the ***energy_virial.dat*** file above. MixPi is currently hard-coded to include every step in the average; if you wish to ignore a set of initial equilibration steps, you can change the value of md%run_average (line number 61 of ***input_setup.F***) to the number of steps to ignore. The columns, from left to right, are time step number, time (in fs), run_temp, run_kinetic, run_potential, and run_total.
- ***energy_actual.dat*** (only when print_level = high): The columns, from left to right are: time step number, time(fs), classical atom kinetic energy, potential energy from cp2k, PI kinetic energy, bead spring potential energy, kinetic virial energy, potential virial energy, kinetic primtive energy

2.2.2 Trajectory Files

MixPI will output trajectory files with system coordinates at a printing frequency specified by the user in ***md_run.inp***. The trajectory files follow the same format as the input initial configuration files (section 2.1.1.1 and Figure 9). Four kinds of trajectory file will be generated, including ***trajectory_run_full.dat***, ***trajectory_run_centroid.dat***, ***trajectory_run_com.dat*** (generated if print_level = high), and ***trajectory_all.dat*** (generated if print_level = high). All trajectories are printed in $\dot{A}$ for positions and $\dot{A}/fs$ for velocities.

- ***trajectory_run_full.dat***: Full trajectories of all particles in the system (All PI beads and all classical atoms).
- ***trajectory_run_centroid.dat***: trajectories of the center of mass of each atom (quantized or not). For example, if a hydrogen atom is quantized with 4 ring polymer beads, it will print the coordinate of the "average hydrogen" as the center of mass of the 4 beads.
- ***trajectory_run_com.dat***: Full trajectories of all particles in the system, with system center of mass at (0,0,0).
- ***trajectory_all.dat***: Full trajectories of all particles including their velocities at each configuration. The first three columns are the positions (in Angstrom) for three dimensions and the second three columns are the velocities (in Angstrom/fs) for three dimensions. (not centered)

2.2.3 Final Configuration Files and RESTARTs

For the user's convenience to restart from the final frame of a previous simulation, final position and velocity files are generated at the end of a simulation. Note, the restart files are printed and read in with the following units: positions in Angstrom ($\dot{A}$) and velocities in $\dot{A}/fs$.

- ***final_atom-vel.xyz*** prints out classical atom velocities in atomic units
- ***final_rp-vel.xyz*** prints out ring polymer bead velocities in atomic units
- ***final_structure.xyz*** prints out the final configuration of all particles (including classical atoms and ring polymer beads) in $\dot{A}$

To restart from the final configuration and velocity at the end of a previous simulation, first change the input file ***md_run.inp*** to read in the initial velocities:

Change line 12 in ***md_run.inp*** (init_vel_rp) to *.false.*

Then, simply change the name of the final position and velocity files (section 2.2.3) to the corresponding input file names:

cp final_atom-vel.xyz atom-vel.dat
cp final_rp-vel.xyz rp-vel.dat
cp final_structure.xyz RP_structure.xyz

Restart your simulation with these files in your working directory.

2.2.4 Other Files

- ***forces.dat*** (generated if print_level = high). This prints the forces (in atomic units: E_h/a_0) in an xyz format for each particle in the system at a frequency specified by the print_freq in ***md_run.inp***.
- ***runtime_info.dat***: A file that contains information about the time it takes to run each step, and the total time takes to complete the simulation. The time each component of MixPI contributes to the calculation time of each step is also shown. Time is in seconds. See Fig. 16.
- cp2k output file: File name is specified by the user in the ***file_info.inp*** input file. This file contains any print statements or error messages generated by CP2K.
- MixPi output to screen: General updates on the progression of the MixPi simulation are printed to the screen.

time for thermostat step	5.40E-02
time for update force env step	0.00E+00
time for calc_force_cp2k step	7.90E-02
time for calc_force_pi step	0.00E+00
time for propagate normal mode step	1.30E-02
time for one step	0.4740
time for one and two step	0.4650
complete run time is	285.0000

Figure 16: An example of the runtime_info.dat file that is printed by MixPI. It provides information on the time it takes to run various subroutines (thermostat, force updates, calculate forces, and propagate normal modes) as well as the time it takes to run an individual step, and the complete run time. It will print the subroutine and one-step timing information for the first 100 steps in a calculation.

3 Analysis Scripts

3.1 RDF calculation

Here, we provide a script (***pi_rdf.F***) to calculate the radial distribution function between different atoms from trajectories of an equilibrated run. We recognize that in a MixPI simulation atoms can be quantized with different number of beads. To calculate the rdf between two atoms involves taking the average of

n_{max} number of rdfs between beads of the two atoms. For example, in the example simulation of a box a water with 4 bead H atoms and 2 bead O atoms, $n_{max} = 4$; therefore, we calculate four rdfs between the hydrogen beads and oxygen beads. These rdfs will follow the force selection rules outlined in Eq. 3. These four rdfs are then averaged together to generate a single rdf that is representative of the entire simulation. Example calculations using this script for rdf calculation are provided in the **example** folders. Three input files are needed to generate the rdf using the script:

- The trajectory file generated from the PIMD simulation for which you want to calculate the rdf. The file must be in xyz format
- ***rp_molecule_info.inp***: This file contains information on the system composition about which molecules contain atoms treated as path integrals and the number of beads for each of these atoms. This file is also used by MixPI and is explained in detail in Sec. 2.1.1. However, the original ***rp_molecule_info.inp*** file needs to be modified to include a description of the classical molecules as 1-bead path-integrals. An example of the ***rp_molecule_info.inp*** used for the M^{3+} + PI e^- system is shown in Fig. 17 (and can be compared to the original version in Fig. 2b).
- ***sim_details.inp***: This file contains system related information that is required for the rdf calculation. An example is shown in 18 with comments explaining each item.

```
3                       !no. of molecules (total)
2   1   electron.psf    !no. of atoms in molecule type; no. of molecule types in system; PSF file
    8   0.02            !no. RP beads atom 1; kinetic mass (not needed)
    1   55.845          !no. RP beads atom 2; kinetic mass (not needed)
3   2   Water.spf
    1   15.998          !1 bead oxygen (classical)
    1   1.0098          !1 bead hydrogen (classical)
    1   1.0098
```

Figure 17: An example of the rp_molecule_info.inp with classical molecules shown as 1-bead PIs. The classical information is added to rp_molecule_info.inp when using the analysis scripts.

```
traj.xyz                !name of trajectory file
200                     !total number of configs in traj file
20                      !number of initial configs to skip
0                       !number of configs to skip between each evaluation
14.0 14.0 14.0          !box size (Angstrom)
HW HW                   !atom 1 and atom 2 label (matches labels in traj.xyz)
0.01 14.0               !xmin and xmax (Angstrom)
```

Figure 18: The input file sim_details.inp that is used by the analysis scripts to calculate radial distribution functions.

4 Theory

Here we review the theory behind PIMD and MTS-PIMD; similar derivations have been presented previously [6, 1]. One particular deviation from some previous derivations is the location of the $\frac{1}{N}$ term, where N is the number of RP beads. In many prior derivations, this term was combined with the β term and, ultimately, resulted in simulations that are effectively conducted at higher temperatures. Since we can have different bead number N for different ring polymers in our mixTS derivation, we derive the RP Hamiltonian to include the $\frac{1}{N_i}$ terms, where i is the label for the ring polymer.

4.1 All Replica Ring Polymer Hamiltonian

The quantum mechanical canonical partition function can be written as

$$\begin{aligned} Z &= \mathrm{Tr}[e^{-\beta \hat{H}}] \\ &= \int dq \ \langle q| \, e^{-\beta \hat{H}} \, |q\rangle \, , \end{aligned} \tag{11}$$

where $|q\rangle$ is a position eigenstate of the one-dimensional Hamiltonian

$$\hat{H} = T(\hat{p}) + V(\hat{q}). \tag{12}$$

We use the asymmetric Trotter approximation to split the Hamiltonian into a potential and kinetic components and rewrite the partition function as

$$Z = \lim_{N\to\infty} \int dq \ \langle q| \left(e^{-\frac{\beta}{N} V(\hat{q})} e^{-\frac{\beta}{N} T(\hat{p})} \right)^N |q\rangle \, , \tag{13}$$

By inserting N-1 copies of the identity in the position basis ($\hat{I} = \int dq \ |q\rangle \langle q|$), we obtain the following expression.

$$Z = \lim_{N\to\infty} \int d\{\mathbf{q}\} \prod_{\alpha=1}^{N} \langle q_\alpha| \left(e^{-\frac{\beta}{N} V(\{\mathbf{q}\})} e^{-\frac{\beta}{N} T(\{\mathbf{p}\})} \right) |q_{\alpha+1}\rangle \, , \tag{14}$$

where $\int d\{\mathbf{q}\} = \int dq_1 \, dq_2 \, \ldots \, dq_{N+1}$, and $q_{N+1} = q_1$ to maintain the trace.

The potential term can be evaluated in the position basis:

$$\langle q_\alpha| \left(e^{-\frac{\beta}{N} V(\{q\})} e^{-\frac{\beta}{N} T(\{p\})} \right) |q_{\alpha+1}\rangle = e^{-\frac{\beta}{N} V(q_\alpha)} \langle q_\alpha| \left(e^{-\frac{\beta}{N} T(\{p\})} \right) |q_{\alpha+1}\rangle \tag{15}$$

In order to evaluate the kinetic term, we introduce identity in the momentum basis and evaluate the inner product ($\langle q|p\rangle = \frac{1}{\sqrt{2\pi\hbar}} e^{\frac{i}{\hbar} pq}$) to obtain

$$\begin{aligned} \langle q_\alpha| \left(e^{-\frac{\beta}{N} T(\{p\})} \right) |q_{\alpha+1}\rangle &= \int dp \, \langle q_\alpha| \, e^{-\frac{\beta}{N} T(\{p\})} \, |p\rangle \, \langle p| \, |q_{\alpha+1}\rangle \\ &= \frac{1}{\sqrt{2\pi\hbar}} e^{-\frac{\beta}{N} T(p)} \int dp \langle q_\alpha|p\rangle e^{-\frac{i}{\hbar} p q_{\alpha+1}} \\ &= \frac{1}{2\pi\hbar} \int dp e^{-\frac{\beta}{N}\left(\frac{p^2}{2m}\right)} e^{\frac{i}{\hbar} p(q_\alpha - q_{\alpha+1})} \end{aligned} \tag{16}$$

This integral can be solved by completing the square to obtain

$$\langle q_\alpha| \left(e^{-\frac{\beta}{N} V(\{q\})} e^{-\frac{\beta}{N} T(\{p\})} \right) |q_{\alpha+1}\rangle = \left(\frac{1}{2\pi\hbar} \right) \left(\frac{2\pi m N}{\beta} \right)^{1/2} e^{-\frac{\beta}{N} V(q_\alpha)} e^{-\frac{Nm}{2\beta\hbar^2}(q_\alpha - q_{\alpha+1})} \tag{17}$$

We can insert this solution into the above expression for the partition function and obtain

$$Z = \lim_{N\to\infty} \left(\frac{mN}{2\pi\beta\hbar^2} \right)^{\frac{N}{2}} \int d\{\mathbf{q}\} e^{-\beta \sum_{\alpha=1}^{N} \left(\frac{1}{N} V(q_\alpha) + \frac{Nm}{2\beta^2\hbar^2} (q_\alpha - q_{\alpha+1})^2 \right)} \tag{18}$$

We now introduce a momentum term by multiplying the above partition function by identity via N Gaussian integrals over momentum with each of them divided by itself for each bead

$$1 = \frac{\int dp_\alpha e^{-\frac{\beta}{N} \frac{p_\alpha^2}{2m_f}}}{\sqrt{\frac{2m_f \pi N}{\beta}}} \tag{19}$$

. We can rewrite the partition function as

$$Z = \lim_{N\to\infty} \left(\frac{\beta}{2\pi m_f N} \right)^{N/2} \left(\frac{mN}{2\pi\beta\hbar^2} \right)^{\frac{N}{2}} \int d\{\mathbf{q}\} d\{\mathbf{p}\} e^{-\beta H_{\mathrm{RP,N}}} \tag{20}$$

.

We now have an expression in which the exponential looks similar to a classical Hamiltonian of the form

$$H_{\mathrm{RP},0} = \frac{1}{N}\sum_{\alpha=1}^{N}\left(\frac{p_\alpha^2}{2m_f} + V(q_\alpha) + \frac{N^2 m}{2\beta^2\hbar^2}(q_\alpha - q_{\alpha+1})^2\right) \tag{21}$$

which is referred to as the one-dimensional PIMD Hamiltonian. For equilibrium properties, one can choose any quantity as m_f, the fictitious mass. The choice of m_f can be used to improve the sampling efficiency of PIMD.

4.2 MixTS Treatment of 1/N

The choice of where to include the $1/N$ term in Eq. 20 can influence the dynamics of a RPMD simulation. Traditionally, the $1/N$ term was often included in the β, which is why many RPMD simulations are considered "high temperature" simulations. When considered a mixed-time slicing (MixTS) approach for MixPI, we do not have a universal $1/N$ that can be factored out of the Hamiltonian. Therefore, we must considered how to accurately distribute the $1/N$ factor into the expression of the Hamiltonian while maintaining the same Boltzmann distributions and dynamics as obtained from Eq. 20.

First, we defined a Hamiltonian that matches the traditional derivation of the RP Hamiltonian in which a $\frac{1}{N}$ term can be brought outside the Hamiltonian in Eq. 21. A second Hamiltonian, in which this term is factored inside the Hamiltonian is written

$$\begin{aligned} H_{\mathrm{RP,N}} &= \sum_{\alpha=1}^{N}\frac{p_\alpha^2}{2Nm_f} + \frac{1}{N}V(q_\alpha) + \frac{Nm}{2\beta^2\hbar^2}(q_\alpha - q_{\alpha+1})^2 \\ Z &= \lim_{N\to\infty}\left(\frac{1}{2\pi\hbar}\right)^N\left(\frac{m}{m_f}\right)^{\frac{N}{2}}\int d\{\mathbf{q}\}d\{\mathbf{p}\}e^{-\beta H_{\mathrm{RP,N}}} \end{aligned} \tag{22}$$

It was previous determined by the Ananth group [2] that the exact treatment of the $\frac{1}{N}$ term is important when generating accurate distributions and dynamics. The expression for a quantum observable $\hat{A}(t)$ using the two different Hamiltonian definitions

$$\langle\hat{A}(t)\rangle_0 = \frac{1}{Z}\int d\{\mathbf{q}\}d\{\mathbf{p}\}e^{-\frac{\beta}{N}H_{\mathrm{RP},0}(\{q(0)\}\{p(0)\})}A(\{q_t\},\{p_t\}) \tag{23}$$

$$\langle\hat{A}(t)\rangle_{\mathrm{N}} = \frac{1}{Z}\int d\{\mathbf{q}\}d\{\mathbf{p}\}e^{-\beta H_{\mathrm{RP,N}}(\{q(0)\}\{p(0)\})}A(\{q_t\},\{p_t\}) \tag{24}$$

. The equations of motion generated for each of these Hamiltonians

$$\begin{aligned} H_0 : \dot{p} &= -\frac{dV}{dq} \\ \dot{q} &= \frac{p}{m} \\ H_{\mathrm{N}} : \dot{p} &= -\frac{1}{N}\frac{dV}{dq} \\ \dot{q} &= -\frac{p}{Nm} \end{aligned} \tag{25}$$

are different depending on whether the $\frac{1}{N}$ term is in the Hamiltonian or is included in the β term. Since the position and momentum initial distributions sampled for $\langle\hat{A}\rangle_{RP}$ and $\langle\hat{A}\rangle_0$are the same, the overall dynamics for H_0 and H_R will only match if equations of motions are also the same. If we look at the corresponding the Langrangian's

$$\begin{aligned} L_0 &= \sum_{\alpha=1}^{N}\frac{mv_\alpha^2}{2} - V(q_\alpha) - \frac{N^2 m}{2\beta^2\hbar^2}(q_\alpha - q_{\alpha+1})^2 \\ L_N &= \sum_{\alpha=1}^{N}\frac{mv_\alpha^2}{N2} - \frac{1}{N}V(q_\alpha) - \frac{Nm}{2\beta^2\hbar^2}(q_\alpha - q_{\alpha+1})^2 \end{aligned} \tag{26}$$

and derive the Euler-Langrange EOM

$$\frac{\partial L}{\partial q} = \frac{d}{dt}\frac{\partial L}{\partial \dot{q}}$$

$$\begin{aligned} L_0 &: -\frac{\partial V}{\partial q_\alpha} - \frac{N^2 m}{\beta^2\hbar^2}(2q_\alpha - q_{\alpha-1} - q_{\alpha+1}) = m\ddot{q_\alpha} \\ L_N &: -\frac{1}{N}\frac{\partial V}{\partial q_\alpha} - \frac{Nm}{\beta^2\hbar^2}(2q_\alpha - q_{\alpha-1} - q_{\alpha+1}) = \frac{m\ddot{q_\alpha}}{N} \end{aligned} \tag{27}$$

we determine both treatments have the same Euler-Langrage EOM. If we use the Langrage definition for momentum

$$p = \frac{\partial L}{\partial \dot{q}} \tag{28}$$

and convert our Langragians back to Hamiltonians

$$\begin{aligned} H &= p\dot{q} - L \\ H_{\mathrm{RP},0} &= \sum_{\alpha=1}^{N} m\dot{q_\alpha}\dot{q_\alpha} - \left(\sum_{\alpha=1}^{N}\frac{mv_\alpha^2}{2} - \frac{N^2 m}{2\beta^2\hbar^2}(q_\alpha - q_{\alpha+1})^2 - V(q_\alpha)\right) \\ &= \sum_{\alpha=1}^{N}\frac{p_\alpha^2}{2m} + V(q_\alpha) + \frac{N^2 m}{2\beta^2\hbar^2}(q_\alpha - q_{\alpha+1})^2 \\ H_{\mathrm{RP,N}} &= \sum_{\alpha=1}^{N}\frac{m\dot{q_\alpha}}{N}\dot{q_\alpha} - \left(\sum_{\alpha=1}^{N}\frac{mv_\alpha^2}{2N} - \frac{Nm}{2\beta^2\hbar^2}(q_\alpha - q_{\alpha+1})^2 - \frac{1}{N}V(q_\alpha)\right) \\ &= \sum_{\alpha=1}^{N}\frac{p_\alpha^2}{2m'} + \frac{1}{N}V(q_\alpha) + \frac{N^2 m'}{2\beta^2\hbar^2}(q_\alpha - q_{\alpha+1})^2 \end{aligned} \tag{29}$$

where $m' = m/N$. The expression for the kinetic piece in H_{RP} has changed from the initial expression by using a different mass. This new Hamiltonian reproduces the same equations of motions as H_0. Therefore, MixPI chooses to use this Hamiltonian in which both the kinetic and harmonic mass terms are scaled ($m' = m/N$).

4.3 MTS-Expansion in Three-Dimensions

Here, we derive the MTS-PIMD Hamiltonian using a similar scheme as in 4.1 . We derive the Hamiltonian for three particles in one-dimension to emphasize how multiple quantization levels interact in the MTS-PIMD Hamiltonian. This Hamiltonian can be extended to handle an arbitrary number of degrees. We start with the same quantum Hamiltonian operator, written for three particles,

$$\begin{aligned} \hat{H} &= \sum_{i=1}^{3}\frac{\hat{p_i}}{2m_i} + \sum_{i=1}^{3}V_1(\hat{q_i}) + \sum_{i=1}^{3}\sum_{j<i}V_2(\hat{q_i},\hat{q_j}) + \sum_{i=1}^{3}\sum_{j<i}\sum_{k<j}V_3(\hat{q_i},\hat{q_j},\hat{q_k}) \\ &= H_1(\hat{q_1},\hat{q_2},\hat{q_3}) + H_2(\hat{q_2},\hat{q_3}) + H_3(\hat{q_3}) \end{aligned} \tag{30}$$

where $\hat{p_i}$, $\hat{q_j}$, and m_i are the momentum, position, and mass of the ith quantum particle. $V_1(\hat{q_i})$ is the one-body portion of the potential energy for the ith degree while $V_2(\hat{q_i},\hat{q_j})$ and $V_3(\hat{q_i},\hat{q_j},\hat{q_k})$ are the two-body and three-body potential energy contributions, respectively. We assume that the q_1 degree of freedom experiences the largest quantum mechanical effect, followed by q_2 and then q_3 and define N_1, N_2, and N_3 as the number of time slices along each degre with $N_1 \geq N_2 \geq N_3$e. We split the Hamiltonian by into components based on the the highest quantized variable a term depends on. For example, $H_1(\hat{q_1},\hat{q_2},\hat{q_3}) = T_1(\hat{p_1}) + V(\hat{q_1}) + V(\hat{q_1},\hat{q_2}) + V(\hat{q_1},\hat{q_3}) + V(\hat{q_1},\hat{q_2},\hat{q_3})$. We use the asymmetric Trotter approximation on this partitioned Hamiltonian to first split H_3 from H_{12}

$$Z = \lim_{N_3\to\infty}\int d\{q\}\,\langle q_1q_2q_3|\left(e^{-\frac{\beta H_3}{N_3}}e^{-\frac{\beta H_{12}}{N_3}}\right)^{N_3}|q_1q_2q_3\rangle \tag{31}$$

and then apply the Trotter approximation twice more to $H_{12} = H_1 + H_2$ and, finally, to H_1 to obtain

$$Z = \lim_{N_3,N_2,N_1\to\infty}\int d\{q\}\,\langle q_1q_2q_3|\left(e^{-\frac{\beta H_3}{N_3}}\left(e^{-\frac{\beta H_2}{N_3N_2}}e^{-\frac{\beta H_1}{N_3N_2}}\right)^{N_2}\right)^{N_3}|q_1q_2q_3\rangle \tag{32}$$

$$Z = \lim_{N_3,N_2,n_1\to\infty} \int d\{q\} \left\langle q_1 q_2 q_3 \right| \left(e^{-\frac{\beta H_3}{N_3}} \left(e^{-\frac{\beta H_2}{N_2}} \left(e^{-\frac{\beta H_1}{N_1}} \right)^{\frac{N_1}{N_2}} \right)^{\frac{N_2}{N_3}} \right)^{N_3} \left| q_1 q_2 q_3 \right\rangle \tag{33}$$

We define $\Lambda_i = e^{-\frac{\beta H_i}{N_i}}$ and $\Lambda_{ij} = \Lambda_i \Lambda_j^{\frac{N_j}{N_i}}$ and rewrite the above expression as

$$Z = \lim_{N_3,N_2,N_1\to\infty} \int d\{q\} \left\langle q_1 q_2 q_3 \right| \left(\Lambda_3 \left(\Lambda_{21} \right)^{\frac{N_2}{N_3}} \right)^{N_3} \left| q_1 q_2 q_3 \right\rangle . \tag{34}$$

. Following a similar serious of steps as employed in the all-replica derivation in the previous section, we insert N_3 copies of identity with respect to the position basis q_3 and evaluate Λ_{12} since it is diagonal with respect to q_3.

$$Z = \lim_{N_3,N_2,N_1\to\infty} \int d\{q\} \left\langle q_1 q_2 \right| \prod_{\alpha=1}^{N_3} \left\langle q_{3,\alpha} \right| \Lambda_3 \left| q_{3,\alpha+1} \right\rangle \Lambda_{12}(q_{3,\alpha+1})^{\frac{N_2}{N_3}} \left| q_1 q_2 \right\rangle . \tag{35}$$

We now have the one-particle results with H_3 and can use the result of the previous derivation to obtain

$$\begin{aligned} Z = \lim_{N_3,N_2,N_1\to\infty} & \left(\frac{\beta}{2\pi m_{3,f}} \right)^{1/2} \left(\frac{m_3 N_3}{2\pi\beta\hbar^2} \right)^{\frac{N_3}{2}} \\ & \int d\{q\} e^{-\beta \sum_{\alpha=1}^{N_3} \left(\frac{p_{3,\alpha}^2}{2m_{3,f}} + \frac{1}{N_3} V_1(q_{3,\alpha}) - \frac{N_3 m_3}{2\beta^2\hbar^2} (q_{3,\alpha} - q_{3,\alpha+1})^2 \right)} \\ & \left\langle q_1 q_2 \right| \prod_{\alpha=1}^{N_3} \Lambda_{12}(q_{3,\alpha+1})^{\frac{N_2}{N_3}} \left| q_1 q_2 \right\rangle . \end{aligned} \tag{36}$$

Next, we introduce N_3 copies of identity with respect to q_2 between each of the $\Lambda(q_{3,\alpha+1})^{N_2/N_3}$ terms

$$\begin{aligned} Z = \lim_{N_3,N_2,N_1\to\infty} A_3 & \int d\{q\} e^{-\beta H_{3,RP}(q_3)} \\ & \left\langle q_1 \right| \prod_{\alpha=1}^{N_3} \left\langle q_{2,(\alpha-1)*N_2/N_3+1} \right| \Lambda_{12}(q_{3,\alpha+1})^{\frac{N_2}{N_3}} \left| q_{2,\alpha*N_2/N_3+1} \right\rangle \left| q_1 \right\rangle . \end{aligned} \tag{37}$$

Since q_2 is more quanitzed than q_3, and needs a greater number of slices then q_3, we introduce additional N_2/N_3 copies of identity between each $\Lambda_{12}(q_{3,\alpha+1})$

$$\begin{aligned} Z = \lim_{N_3,N_2,N_1\to\infty} A_3 & \int d\{q\} e^{-\beta H_{3,RP}(q_3)} \left\langle q_1 \right| \\ & \prod_{\alpha=1}^{N_3} \prod_{\gamma=1}^{N_2/N_3} \left\langle q_{2,(\alpha-1)*N_2/N_3+\gamma} \right| \Lambda_2(q_{3,\alpha+1}) \Lambda_1(q_{3,\alpha+1})^{\frac{N_1}{N_2}} \left| q_{2,(\alpha-1)*N_2/N_3+\gamma+1} \right\rangle \left| q_1 \right\rangle . \end{aligned} \tag{38}$$

$\Lambda_1(q_{3,\alpha+1})$ is diagonal with respect to q_2 and we have obtained the one-dimensional ring-polymer solution for q_2.

$$\begin{aligned} Z = \lim_{N_3,N_2,N_1\to\infty} A_3 A_2 & \int d\{q\} e^{-\beta H_{3,RP}(\{q_3\})} e^{-\beta H_{2,RP}(\{q_2\},\{q_3\})} \\ & \prod_{\alpha=1}^{N_3} \prod_{\gamma=1}^{N_2/N_3} \left\langle q_1 \right| \Lambda_1(q_{3,\alpha+1}, q_{2,(\alpha-1)*N_2/N_3+\gamma+1})^{\frac{N_1}{N_2}} \left| q_1 \right\rangle . \end{aligned} \tag{39}$$

We follow the same procedure to introduce N_2 copies of identity between each $\Lambda_1(q_{3,\alpha+1}, q_{2,(\alpha-1)*N_2/N_3+\gamma+1})^{\frac{N_1}{N_2}}$ and N_1/N_2 copies of identity between each $\Lambda_1(q_{3,\alpha+1}, q_{2,(\alpha-1)*N_2/N_3+\gamma+1}$ to obtain the all-replica Hamiltonian for q_1. Once again, we introduce N_2 copies between each of the copies of $\Lambda_1(q_{3,\alpha+1}, q_{2,(\alpha-1)*N_2/N_3+\gamma+1})^{\frac{N_1}{N_2}}$

$$\begin{aligned} Z = \lim_{N_3,N_2,N_1\to\infty} A_3 A_2 & \int d\{q\} e^{-\beta H_{3,RP}(\{q_3\})} e^{-\beta H_{2,RP}(\{q_2\},\{q_3\})} \\ & \prod_{\alpha=1}^{N_3} \prod_{\gamma=1}^{N_2/N_3} \left\langle q_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+1} \right| \Lambda_1(q_{3,\alpha+1}, q_{2,(\alpha-1)*N_2/N_3+\gamma+1})^{\frac{N_1}{N_2}} \\ & \left| q_{1,((\alpha-1)*N_2/N_3+\gamma)*N_1/N_2+1} \right\rangle . \end{aligned} \tag{40}$$

as well as N_1/N_2 copies of identity in q_1 between each $\Lambda_1(q_{3,\alpha+1}, q_{2,(\alpha-1)*N_2/N_3+\gamma+1}$

$$
\begin{aligned}
Z = \lim_{N_3,N_2,N_1\to\infty} A_3 A_2 \int d\{q\} e^{-\beta H_{3,RP}(\{q_3\})} e^{-\beta H_{2,RP}(\{q_2\},\{q_3\})} \\
\prod_{\alpha=1}^{N_3} \prod_{\gamma=1}^{N_2/N_3} \prod_{\lambda=1}^{N_1/N_2} \langle q_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+\lambda} | \, \Lambda_1(q_{3,\alpha+1}, q_{2,(\alpha-1)*N_2/N_3+\gamma+1}) \\
| q_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+\lambda+1} \rangle \, .
\end{aligned}
\tag{41}
$$

We obtain an expression for the three-level mixed-time slicing RPMD partition function and Hamiltonian

$$
Z = \lim_{N_3,N_2,N_1\to\infty} A_3 A_2 A_1 \int d\{q\} e^{-\beta H_{3,RP}(\{q_3\})} e^{-\beta H_{2,RP}(\{q_2\},\{q_3\})} e^{-\beta H_{1,RP}(\{q_1\},\{q_2\},\{q_3\})}. \tag{42}
$$

where

$$
\begin{aligned}
H_{3,RP} &= \sum_{\alpha=1}^{N_3} \frac{p_{3,\alpha}^2}{2m_{3,f}} + \frac{1}{N_3} V_1(q_{3,\alpha}) - \frac{N_3 m_3}{2\beta^2\hbar^2} (q_{3,\alpha} - q_{3,\alpha+1})^2 \\
H_{2,RP} &= \sum_{\alpha=1}^{N_3} \sum_{\gamma=1}^{N_2/N_3} \frac{p_{2,(\alpha-1)*N_2/N_3+\gamma}^2}{2m_2} + \frac{1}{N_2} V_1(q_{2,(\alpha-1)*N_2/N_3+\gamma} + \frac{1}{N_2} V_2(q_{2,(\alpha-1)*N_2/N_3+\gamma}, q_{3,\alpha}) \\
&\quad + \frac{N_2 m_2}{2\beta^2\hbar^2} (q_{2,(\alpha-1)*N_2/N_3+\gamma} - q_{2,(\alpha-1)*N_2/N_3+\gamma+1})^2 \\
H_{1,RP} &= \sum_{\alpha=1}^{N_3} \sum_{\gamma=1}^{N_2/N_3} \sum_{\lambda=1}^{N_1/N_2} \frac{p_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+\lambda}^2}{2m_1} + \frac{1}{N_1} V_1(q_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+\lambda}) \\
&\quad + \frac{1}{N_1} V_2(q_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+\lambda}, q_{2,(\alpha-1)*N_2/N_3+\gamma}) \\
&\quad + \frac{1}{N_1} V_2(q_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+\lambda}, q_{3,\alpha}) \\
&\quad + \frac{1}{N_1} V_3(q_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+\lambda}, q_{2,(\alpha-1)*N_2/N_3+\gamma}, q_{3,\alpha}) \\
&\quad + \frac{N_3 m_3}{2\beta^2\hbar^2} (q_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+\lambda} - q_{1,((\alpha-1)*N_2/N_3+\gamma-1)*N_1/N_2+\lambda+1})^2
\end{aligned}
\tag{43}
$$

This Hamiltonian can also be rewritten as

$$
H_{\mathrm{RP}}^{\mathrm{mix}} = H_0 + V_{\mathrm{mix}} \tag{44}
$$

$$
H_0 = \sum_{i=1}^{3} \sum_{\alpha=1}^{N_i} \frac{p_{i,\alpha}^2}{2m_i} + \frac{N_i m_i}{2\beta^2\hbar^2} (q_{i,\alpha} - q_{i,\alpha+1})^2 \tag{45}
$$

$$
\begin{aligned}
V_{\mathrm{mix}} = \sum_{i=1}^{3} \frac{1}{N_i} \Bigg(\sum_{\alpha=1}^{N_i} V_i(q_{i,\alpha}) + \\
\sum_{j>i} \sum_{\gamma=1}^{\frac{N_i}{N_j}} \Bigg(\sum_{\alpha=1}^{N_j} V_{ij}(q_{i,(\alpha-1)\frac{N_i}{N_j}+\gamma}, q_{j,\alpha}) + \\
\sum_{k>j} \sum_{\alpha=1}^{N_k} \sum_{\lambda=1}^{\frac{N_j}{N_k}} V_{ijk}(q_{i,((\alpha-1)\frac{N_j}{N_k}+\lambda-1)\frac{N_i}{N_j}+\gamma}, q_{j,(\alpha-1)\frac{N_j}{N_k}+\lambda}, q_{k,\alpha}) \Bigg) \Bigg) \Bigg) ,
\end{aligned}
\tag{46}
$$

5 Funding Acknowledgment

The authors are grateful to Greg Schenter for useful discussions. B.A.J. and C.J.M acknowledge support by the DOE Office of Science, Office of Basic Energy Sciences, Division of Chemical Sciences, Geosciences,

and Biosciences, Condensed Phase and Interfacial Molecular Science program, FWP 16249. PNNL is operated by Battelle Memorial Institute for the United States Department of Energy under DOE Contract No. DE-AC05-76RL1830. Research reported in this publication was also partially supported by the National Institute of General Medical Sciences of the National Institutes of Health under award number 1R35GM158400-01 to N.A., S. B. thanks the Cornell Arts and Sciences New Frontiers Grant for seed funding. In addition, N.A. and B.A.J. acknowledge prior support from the U.S. Department of Energy, Office of Basic Energy Sciences, Division of Chemical Sciences, Geosciences and Biosciences under Award DE-FG02-12ER16362 (Nanoporous Materials Genome: Methods and Software to Optimize Gas Storage, Separations, and Catalysis). This research used resources of the National Energy Research Scientific Computing Center (NERSC), a Department of Energy User Facility using NERSC award BES-ERCAP0036077.

References

[1] Michele Ceriotti, Michele Parrinello, Thomas E. Markland, and David E. Manolopoulos. Efficient stochastic thermostatting of path integral molecular dynamics. *J. Chem. Phys*, 133(12), 2010.

[2] Elliot Eklund. *Path Integral Computational Methods for Condensed Phase Systems: From Classical to Quantum*. Thesis, Cornell University, 2022.

[3] B. A. Johnson, S. Bu, and N. Ananth. MixPI: Software for atomistic PIMD simulations with mixed time slicing. *https://github.com/AnanthGroup/MixPI*, June 2026.

[4] Rachel L. Kenion and Nandini Ananth. Direct simulation of electron transfer in the cobalt hexammine(II/III) self-exchange reaction. *Phys. Chem. Chem. Phys.*, 18(37):26117–26124, 2016.

[5] Thomas D. Kühne, Marcella Iannuzzi, Mauro Del Ben, Vladimir V. Rybkin, Patrick Seewald, Frederick Stein, Teodoro Laino, Rustam Z. Khaliullin, Ole Schütt, Florian Schiffmann, Dorothea Golze, Jan Wilhelm, Sergey Chulkov, Mohammad Hossein Bani-Hashemian, Valéry Weber, Urban Borštnik, Mathieu Taillefumier, Alice Shoshana Jakobovits, Alfio Lazzaro, Hans Pabst, Tiziano Müller, Robert Schade, Manuel Guidon, Samuel Andermatt, Nico Holmberg, Gregory K. Schenter, Anna Hehn, Augustin Bussy, Fabian Belleflamme, Gloria Tabacchi, Andreas Glöß, Michael Lass, Iain Bethune, Christopher J. Mundy, Christian Plessl, Matt Watkins, Joost VandeVondele, Matthias Krack, and Jürg Hutter. Cp2k: An electronic structure and molecular dynamics software package - quickstep: Efficient and accurate electronic structure calculations. *J. Chem. Phys*, 152(19), 2020.

[6] Ryan P. Steele, Jill Zwickl, Philip Shushkov, and John C. Tully. Mixed time slicing in path integral simulations. *J. Chem. Phys*, 134(7), 2011.